Solution: Problem Challenge 2: Rearrange a LinkedList

Problem Statement

Given the head of a Singly LinkedList, write a method to modify the LinkedList such that the **nodes from the second half of the LinkedList are inserted alternately to the nodes from the first half in reverse order**. So if the LinkedList has nodes `1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null`, your method should return `1 -> 6 -> 2 -> 5 -> 3 -> 4 -> null`.

Your algorithm should use only constant space the input LinkedList should be modified in-place.

Example 1:

```
Input: 2 -> 4 -> 6 -> 8 -> 10 -> 12 -> null
Output: 2 -> 12 -> 4 -> 10 -> 6 -> 8 -> null
```

Example 2:

```
Input: 2 -> 4 -> 6 -> 8 -> 10 -> null
Output: 2 -> 10 -> 4 -> 8 -> 6 -> null
```

Solution

This problem shares similarities with `Palindrome LinkedList`. To rearrange the given LinkedList we will follow the following steps:

1. We can use the **Fast & Slow pointers** method similar to `Middle of the LinkedList` to find the middle node of the LinkedList.
2. Once we have the middle of the LinkedList, we will reverse the second half of the LinkedList.
3. Finally, we'll iterate through the first half and the reversed second half to produce a LinkedList in the required order.

Here is the visual representation of this algorithm for Example-1:

Code

Here is what our algorithm will look like:

Java

```java
/*class ListNode {
  int val = 0;
  ListNode next;

  ListNode(int val) {
    this.val = val;
  }
}*/

class Solution {

  public ListNode reorder(ListNode head) {
    if (head == null || head.next == null)
      return head;

    // find the middle of the LinkedList
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
      slow = slow.next;
      fast = fast.next.next;
    }
```

Run Code Result | Stdout

You must run your code first

Execute

Time Complexity

The above algorithm will have a time complexity of $O(N)$ where N is the number of nodes in the LinkedList.

Space Complexity

The algorithm runs in constant space $O(1)$.